

CO1-ASSESSMENT TOOL-1

Name: R.GAUTHAM KRISHNA

Reg.no: 192421128

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 16-07-2026

Q27. Application: Natural Language Processing Engine

Problem Statement

An NLP engine processes text data using the recurrence:

$$T(n) = T(n/2) + n$$

Apply the substitution method to solve the recurrence. Expand the relation step-by-step and derive the time complexity. Analyze scalability as the text corpus size increases and interpret the result using asymptotic notation.

Objective

- Solve the recurrence using the substitution method.
- Expand the recurrence step-by-step.
- Derive the time complexity.
- Analyze performance as text corpus size increases.

Concept Used

Substitution Method for solving recurrence relations.

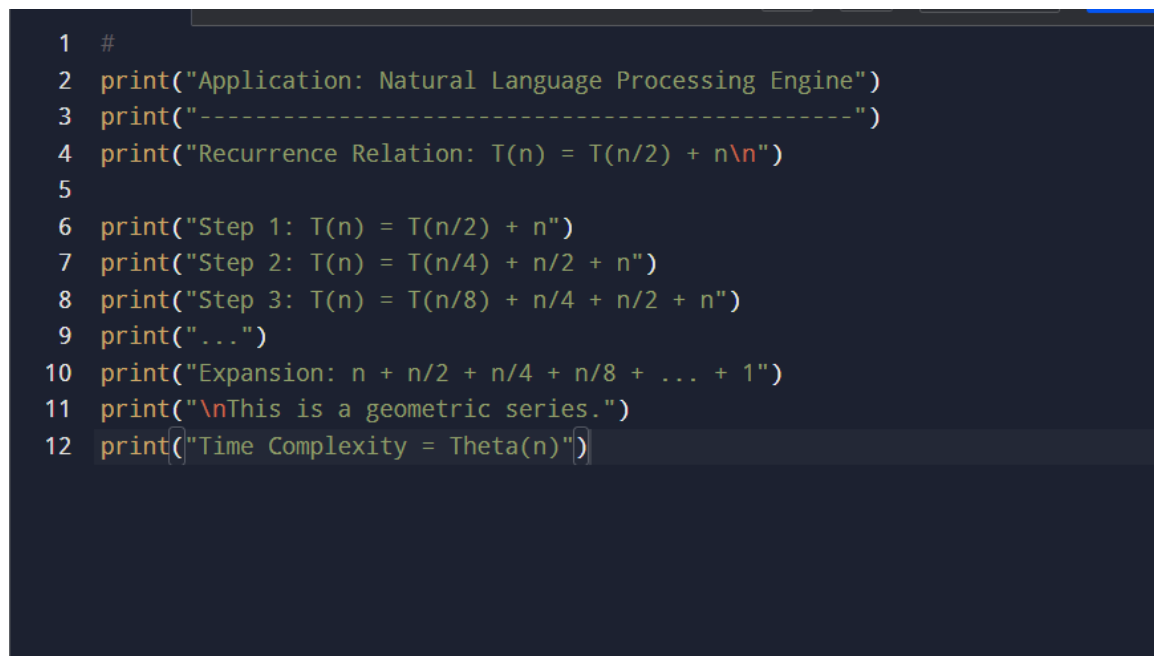
Algorithm / Approach

1. Write the recurrence relation.
2. Expand recursively.
3. Continue until the base case.
4. Sum all the terms.
5. Determine the asymptotic complexity.

Source Code (Python)

```
print("T(n)=T(n/2)+n")
print("Expansion:")
print("= n + n/2 + n/4 + n/8 + ... + 1")
print("Geometric Series Sum = 2n")
print("Time Complexity: Theta(n)")
```

Source Code Screenshot

A screenshot of a code editor with a dark background and light-colored text. The code is a Python script that prints various mathematical expressions and a time complexity analysis. The code is as follows:

```
1 #
2 print("Application: Natural Language Processing Engine")
3 print("-----")
4 print("Recurrence Relation: T(n) = T(n/2) + n\n")
5
6 print("Step 1: T(n) = T(n/2) + n")
7 print("Step 2: T(n) = T(n/4) + n/2 + n")
8 print("Step 3: T(n) = T(n/8) + n/4 + n/2 + n")
9 print("...")
10 print("Expansion: n + n/2 + n/4 + n/8 + ... + 1")
11 print("\nThis is a geometric series.")
12 print("Time Complexity = Theta(n)")
```

Sample Output

T(n)=T(n/2)+n
Expansion:
= n + n/2 + n/4 + n/8 + ... + 1
Geometric Series Sum = 2n
Time Complexity: Theta(n)

Output Screenshot

Application: Natural Language Processing Engine

Recurrence Relation: $T(n) = T(n/2) + n$

Step 1: $T(n) = T(n/2) + n$

Step 2: $T(n) = T(n/4) + n/2 + n$

Step 3: $T(n) = T(n/8) + n/4 + n/2 + n$

...

Expansion: $n + n/2 + n/4 + n/8 + \dots + 1$

This is a geometric series.

Time Complexity = $\Theta(n)$

=== Code Execution Successful ===

Analysis

The recurrence expands into the series:

$$T(n) = n + n/2 + n/4 + n/8 + \dots + 1$$

This is a geometric series whose sum is approximately $2n$.

Therefore,

$$T(n) = \Theta(n)$$

Performance Analysis

- Text size is reduced by half at every recursive step.
- Total work grows linearly with input size.
- Doubling the corpus size roughly doubles the execution time.
- Suitable for processing large text datasets.
- Provides good scalability.

Conclusion

The substitution method shows that the recurrence has a time complexity of $\Theta(n)$. The NLP engine scales efficiently for large text corpora.

Q40. Application: Genome Sequencing Analysis

Problem Statement

A genome sequencing algorithm is modeled as:

$$T(n) = 3T(n/2) + n$$

Apply the Master Theorem to determine the time complexity. Clearly identify the parameters and justify the selected case. Analyze how scalability affects large biological datasets.

Objective

- Analyze the recurrence using the Master Theorem.
- Identify a, b, and f(n).
- Determine the time complexity.
- Analyze scalability for genome sequencing.

Concept Used

Master Theorem for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

Algorithm / Approach

1. Identify a, b, and f(n).
2. Compute $n^{(\log_b a)}$.
3. Compare f(n) with $n^{(\log_b a)}$.
4. Select the appropriate Master Theorem case.
5. Determine the time complexity.

Source Code (Python)

```
import math
```

```
a = 3
```

```
b = 2
```

```
print("T(n)=3T(n/2)+n")
```

```
print("a =", a)
```

```
print("b =", b)
```

```
print("f(n)=n")
```

```
print("n^(log2 3)=n^1.585")
```

```
print("Master Theorem Case: 1")
```

```
print("Time Complexity: Theta(n^1.585)")
```

Source Code Screenshot

```
1 import math
2
3 a = 3
4 b = 2
5
6 print("T(n) = 3T(n/2) + n")
7 print("a =", a)
8 print("b =", b)
9 print("f(n) = n")
10 print("n^(log2 3) = n^", round(math.log(a, b), 3))
11 print("Master Theorem Case: 1")
12 print("Time Complexity: Theta(n^1.585)")
```

Sample Output

$T(n)=3T(n/2)+n$

$a = 3$

$b = 2$

$f(n)=n$

$n^{(\log_2 3)}=n^{1.585}$

Master Theorem Case: 1

Time Complexity: $\Theta(n^{1.585})$

Output Screenshot

```
Output
T(n) = 3T(n/2) + n
a = 3
b = 2
f(n) = n
n^(log2 3) = n^ 1.585
Master Theorem Case: 1
Time Complexity: Theta(n^1.585)

=== Code Execution Successful ===
```

Analysis

- $a = 3$
- $b = 2$
- $f(n) = n$
- $n^{(\log_2 3)} \approx n^{1.585}$

Since

$$f(n) = O(n^{(1.585 - \epsilon)})$$

for $\epsilon > 0$, Master Theorem Case 1 applies.

Therefore,

$$T(n) = \Theta(n^{1.585})$$

Performance Analysis

- Genome data is divided into three recursive subproblems.
- Execution time grows as $\Theta(n^{1.585})$.
- Doubling genome size increases work by approximately $3\times$.
- More efficient than quadratic algorithms.
- Suitable for analyzing moderately large biological datasets.

Conclusion

The recurrence satisfies Master Theorem Case 1. Therefore, the genome sequencing algorithm has a time complexity of $\Theta(n^{1.585})$, providing better scalability than $\Theta(n^2)$ algorithms for large datasets.

